



Universidade Federal de Santa Catarina – UFSC
Centro Tecnológico – CTC
Departamento de Informática e Estatística – INE

INE 5413 – Grafos

Atividade Prática A1

Discente:

Gustavo Monteiro Jorge (24103615)

Ana Luiza Sales Gobbi (24105453)

Florianópolis, 21 de setembro de 2025.

Sumário

1	Introdução	3
2	Algoritmos e estruturas de dados	3
2.1	Representação de grafo: <i>grafo.py</i>	3
2.2	Busca em largura: <i>buscador_em_largura.py</i>	3
2.3	Ciclo Euleriano (ou Hierholzer): <i>ciclo_euleriano.py</i>	3
2.4	Caminhos Mínimos (Dijkstra e Bellman-Ford): <i>dijkstra.py</i> e <i>bellman_ford.py</i>	4
2.5	Caminhos mínimos (Floyd-Warshall): <i>floyd_warshall.py</i>	4

1 Introdução

Este relatório descreve as estruturas de dados selecionadas para a implementação dos algoritmos da Atividade Prática A1. As escolhas foram guiadas pelos requisitos de cada problema, pela busca por eficiência computacional e pela conformidade com os modelos teóricos apresentados na apostila da disciplina.

2 Algoritmos e estruturas de dados

2.1 Representação de grafo: *grafo.py*

O grafo não-dirigido e ponderado foi representado por uma matriz de adjacências e, para isso, escolhemos um dicionário de dicionários (*vertice: vizinho: peso*). Esta estrutura é ideal para a maioria das aplicações práticas, especialmente para grafos esparsos (onde o número de arestas é significativamente menor que o número de vértices ao quadrado). Para isto, existem algumas razões:

- Eficiência de espaço: armazena apenas as arestas existentes, consumindo memória proporcional a $|V| + |E|$.
- Eficiência de tempo: o uso de dicionários (hash maps) em Python permite que operações como verificação de existência de aresta (*haAresta*) e obtenção de peso (*peso*) sejam realizadas em tempo médio de $O(1)$.
- A enumeração de vizinhos de um vértice v tem complexidade proporcional ao seu grau, $O(d(v))$, o que é ótimo para algoritmos de busca.

2.2 Busca em largura: *buscador_em_largura.py*

A Busca em Largura (BFS) explora o grafo por níveis a partir de uma origem. Para isso, utilizamos as estruturas fila (*collections.deque*) e conjunto (*set*). As justificativas são:

- A fila é a estrutura de dados fundamental para o algoritmo BFS, garantindo que os vértices sejam processados na ordem em que são descobertos (FIFO), o que permite a exploração por níveis. A classe deque de Python é altamente otimizada, oferecendo operações de enfileirar (*append*) e desenfileirar (*popleft*) em tempo $O(1)$.
- O conjunto foi utilizado para armazenar os vértices já visitados. Esta escolha é mais eficiente do que uma lista, pois a verificação de pertencimento (*in*) tem complexidade média de $O(1)$, evitando reprocessamento de vértices e garantindo a finalização do algoritmo.

2.3 Ciclo Euleriano (ou Hierholzer): *ciclo_euleriano.py*

Para a implementação do Algoritmo de Hierholzer, que constrói um ciclo Euleriano, a estrutura central foi a pilha, implementada com uma lista.

- O algoritmo de Hierholzer, na versão iterativa, baseia-se em avançar por um caminho até encontrar um "beco sem saída" (um vértice cujas arestas já foram todas visitadas) e, então, retroceder, adicionando os vértices do caminho ao ciclo final. Esse comportamento de "último a entrar, primeiro a sair" (LIFO) é naturalmente modelado por uma pilha. Uma lista em Python, com as operações *push* e *pop*, funciona como uma pilha eficiente com complexidade $O(1)$.

2.4 Caminhos Mínimos (Dijkstra e Bellman-Ford): *dijkstra.py* e *bellman_ford.py*

Antes de tudo, foi implementado em *A1_4.py* que houvesse uma escolha entre os dois algoritmos, baseando-se na presença ou não de pesos negativos. Fizemos isso por eficiência, visto que, apesar de Dijkstra não suportar pesos negativos, possui eficiência maior em muitos casos em comparação a Bellman-Ford.

Para esses algoritmos de caminhos mínimos de fonte única, a estrutura base usada foi uma lista para armazenar os dados de cada nó (*No* para Dijkstra e *BFNo* para Bellman-Ford).

- Dijkstra: A implementação segue o Algoritmo 13 da apostila, que mantém um conjunto de vértices e, a cada passo, busca o nó não conhecido com a menor distância estimada. Uma lista simples foi suficiente para armazenar os objetos *No* (contendo distância, predecessor e booleano "conhecido"). A função *verticeMinimo* itera sobre ela para encontrar o próximo vértice a ser processado. Embora uma fila de prioridade seja mais eficiente, esta abordagem é uma implementação mais direta do pseudocódigo.
- Bellman-Ford: O Algoritmo 12 da apostila exige que todas as arestas do grafo sejam percorridas $|V - 1|$ vezes. Para facilitar essa lógica, uma lista de arestas foi gerada inicialmente. Isso permite iterar de forma simples e direta sobre todas as arestas a cada passo do laço principal.

2.5 Caminhos mínimos (Floyd-Warshall): *floyd_warshall.py*

A matriz é implementada como uma lista de listas. O algoritmo de Floyd-Warshall é um método de programação dinâmica que opera sobre uma matriz de distâncias. A cada iteração k , ele atualiza a distância entre cada par (i, j) com base na possibilidade de usar k como um vértice intermediário. A matriz oferece acesso direto em tempo $O(1)$ a qualquer distância $D[i][j]$, o que é essencial para a eficiência da operação central do algoritmo, resultando na complexidade de tempo $\Theta(|V|^3)$.